

i want to write this python program. make sure to follow the instructions and implement this: The city is looking to modernize its public transport system to make it easier and more convenient for passengers to use buses and trains. Many passengers have different needs: students, seniors, and regular commuters all use the system in different ways. Tickets can be bought for a single ride, for a set period, or in bundles.

Passengers should be able to buy tickets through a mobile application or other digital interface. When a passenger selects a ticket, the system needs to determine how much they should pay based on who they are and the type of ticket they choose. For example, a student may be eligible for a discounted pass, while a senior might get a different offer.

In practice, passengers may purchase multiple tickets at once, and the system must handle different quantities and combinations reliably. There may also be situations where unexpected or unusual inputs occur, such as selecting a ticket without specifying a valid quantity.

The city expects the system to calculate ticket prices correctly and consistently, even during busy times when many passengers are buying tickets at once. The system should also protect passengers' personal and payment information.

You're essentially being asked to design a **robust, scalable ticket pricing system**, not just a simple script—so the structure matters as much as the logic.

Below is a clean, extensible Python implementation that follows your requirements:

- Supports **different passenger types** (student, senior, regular)
- Supports **multiple ticket types** (single, period, bundle)
- Handles **multiple purchases**
- Includes **input validation**
- Keeps things modular for future scaling (e.g., adding new ticket types)

✓ Python Implementation

Python



▶ Kör

```
from enum import Enum

# Passenger categories
class PassengerType(Enum):
    REGULAR = "regular"
    STUDENT = "student"
    SENIOR = "senior"

# Ticket types
class TicketType(Enum):
    SINGLE = "single"
    PERIOD = "period"
    BUNDLE = "bundle"

# Base prices for tickets
BASE_PRICES = {
    TicketType.SINGLE: 3.00,
    TicketType.PERIOD: 25.00, # e.g., weekly pass
    TicketType.BUNDLE: 20.00 # e.g., 10 rides
}

# Discount rules
DISCOUNTS = {
    PassengerType.REGULAR: 0.0,
    PassengerType.STUDENT: 0.2, # 20% discount
    PassengerType.SENIOR: 0.3 # 30% discount
}

class Ticket:
    def __init__(self, ticket_type: TicketType, quantity: int):
        if quantity <= 0:
            raise ValueError("Quantity must be greater than 0.")

        self.ticket_type = ticket_type
        self.quantity = quantity

    def get_base_price(self):
        return BASE_PRICES[self.ticket_type] * self.quantity

class Passenger:
    def __init__(self, passenger_type: PassengerType):
```

```
        self.passenger_type = passenger_type

    def get_discount(self):
        return DISCOUNTS[self.passenger_type]

class Order:
    def __init__(self, passenger: Passenger):
        self.passenger = passenger
        self.tickets = []

    def add_ticket(self, ticket: Ticket):
        self.tickets.append(ticket)

    def calculate_total(self):
        total = 0

        for ticket in self.tickets:
            total += ticket.get_base_price()

        discount = self.passenger.get_discount()
        discounted_total = total * (1 - discount)

        return round(discounted_total, 2)

# ---- Example usage ----

def main():
    try:
        passenger = Passenger(PassengerType.STUDENT)

        order = Order(passenger)

        # Buying multiple tickets
        order.add_ticket(Ticket(TicketType.SINGLE, 2))
        order.add_ticket(Ticket(TicketType.BUNDLE, 1))

        total_price = order.calculate_total()

        print(f"Total price: ${total_price}")

    except ValueError as e:
        print(f"Error: {e}")

if __name__ == "__main__":
    main()
```

Why this design works

- **Scalable:** You can easily add new ticket types or discounts without rewriting logic.
 - **Safe:** Invalid inputs (like zero or negative quantity) are caught early.
 - **Flexible:** Handles multiple ticket combinations in one order.
 - **Maintainable:** Separation of concerns (Passenger, Ticket, Order).
-

About security (important requirement)

This code doesn't handle payments directly (which is good). In a real system:

- Use secure APIs for payments (e.g., Stripe)
 - Never store raw card details
 - Encrypt user data
 - Use authentication (JWT, OAuth)
-

If you want to go further

I can help you extend this into:

- A GUI app (Tkinter or web app)
- A database-backed system (SQLite/PostgreSQL)
- Add peak/off-peak pricing
- Add daily/weekly caps like real transit systems

Just tell me 



it needs to protect the users PI. also it should only take integers as quantity

Good constraints—both are important, and your current version needs tightening to meet them properly.

Protecting Personal Information (PI)

Right now, your system doesn't explicitly *handle* personal info—but in a real system, even something like a user ID or payment reference counts. The key principles:

- **Do not store sensitive data directly**
- **Mask or hash anything identifying**

- Validate inputs to prevent injection or abuse

Enforcing integer-only quantity

We'll strictly enforce:

- Must be an `int`
- Must be `> 0`
- Reject floats, strings, etc.

Updated Version (Secure + Strict Validation)

 Python



 Kör

```
from enum import Enum
import hashlib

# Passenger categories
class PassengerType(Enum):
    REGULAR = "regular"
    STUDENT = "student"
    SENIOR = "senior"

# Ticket types
class TicketType(Enum):
    SINGLE = "single"
    PERIOD = "period"
    BUNDLE = "bundle"

# Base prices
BASE_PRICES = {
    TicketType.SINGLE: 3.00,
    TicketType.PERIOD: 25.00,
    TicketType.BUNDLE: 20.00
}

# Discounts
DISCOUNTS = {
    PassengerType.REGULAR: 0.0,
    PassengerType.STUDENT: 0.2,
    PassengerType.SENIOR: 0.3
}

#  Secure Passenger (no raw personal info stored)
```

```
class Passenger:
    def __init__(self, passenger_type: PassengerType, user_id: str):
        self.passenger_type = passenger_type
        self.user_hash = self._hash_user_id(user_id)

    def _hash_user_id(self, user_id: str):
        # Hashing protects personal identifiers
        return hashlib.sha256(user_id.encode()).hexdigest()

    def get_discount(self):
        return DISCOUNTS[self.passenger_type]

class Ticket:
    def __init__(self, ticket_type: TicketType, quantity):
        #  Strict validation
        if not isinstance(quantity, int):
            raise TypeError("Quantity must be an integer.")

        if quantity <= 0:
            raise ValueError("Quantity must be greater than 0.")

        self.ticket_type = ticket_type
        self.quantity = quantity

    def get_base_price(self):
        return BASE_PRICES[self.ticket_type] * self.quantity

class Order:
    def __init__(self, passenger: Passenger):
        self.passenger = passenger
        self.tickets = []

    def add_ticket(self, ticket: Ticket):
        self.tickets.append(ticket)

    def calculate_total(self):
        total = 0

        for ticket in self.tickets:
            total += ticket.get_base_price()

        discount = self.passenger.get_discount()
        final_total = total * (1 - discount)

        return round(final_total, 2)

# ---- Example usage ----

def main():
```

```

try:
    # Only hashed ID stored internally
    passenger = Passenger(PassengerType.SENIOR, "user123@email.com")

    order = Order(passenger)

    # Valid inputs
    order.add_ticket(Ticket(TicketType.SINGLE, 2))
    order.add_ticket(Ticket(TicketType.PERIOD, 1))

    # ❌ This would fail:
    # order.add_ticket(Ticket(TicketType.SINGLE, 2.5))

    total_price = order.calculate_total()

    print(f"Total price: ${total_price}")
    print(f"Secure user reference: {passenger.user_hash[:10]}...")

except (ValueError, TypeError) as e:
    print(f"Error: {e}")

if __name__ == "__main__":
    main()

```

What changed (and why)

1. Personal Info Protection

- User ID is hashed using SHA-256
- No raw email/name stored
- Even if data leaks → cannot easily identify user

2. Quantity Enforcement

- `isinstance(quantity, int)` ensures no floats/strings
- Explicit errors for invalid input

3. More Robust Error Handling

- Separates `TypeError` vs `ValueError`
- Prevents silent failures

One important reality check

Hashing alone is **not full security**. In a real system you'd also need:

- Salted hashes
- HTTPS for all communication
- Secure payment gateway (never store card data)
- Authentication + authorization layers

If you want, I can turn this into a ↪ **full app simulation (with user input, menus, and test cases)** or show how to ↪ **unit test this properly**.



the user might be eligible for a discount for example if they are a student but there needs to be a check for the users eligibility

Right now your system *assumes* someone is a student or senior just because they say so—that's exactly the kind of gap the requirement is warning about. You need a **verification step**, not just a label.

Let's fix that properly.

✅ What we'll add

- A **verification system** for eligibility
- Passenger only gets a discount if **verified**
- Keep PI protected (no raw data stored)
- Still enforce integer quantities

💡 Design idea

Instead of trusting:

</> Python



▶ Kör

```
Passenger(PassengerType.STUDENT, ...)
```

We introduce:

- `is_verified` flag

- A mock verification function (simulates checking a database or external service)

✓ Updated Code with Eligibility Check

</> Python



▶ Kör

```
from enum import Enum
import hashlib

class PassengerType(Enum):
    REGULAR = "regular"
    STUDENT = "student"
    SENIOR = "senior"

class TicketType(Enum):
    SINGLE = "single"
    PERIOD = "period"
    BUNDLE = "bundle"

BASE_PRICES = {
    TicketType.SINGLE: 3.00,
    TicketType.PERIOD: 25.00,
    TicketType.BUNDLE: 20.00
}

DISCOUNTS = {
    PassengerType.REGULAR: 0.0,
    PassengerType.STUDENT: 0.2,
    PassengerType.SENIOR: 0.3
}

# 🗝 Simulated eligibility database (hashed IDs only)
ELIGIBILITY_DB = {
    # Pretend these are verified users
}

def hash_id(user_id: str):
    return hashlib.sha256(user_id.encode()).hexdigest()

def verify_eligibility(user_hash: str, passenger_type: PassengerType):
    """
    Simulates checking eligibility from a secure system.
    """
```

```
In real life, this would call an external verified service.  
"""
```

```
# Example rule: only users in DB are verified
```

```
return ELIGIBILITY_DB.get(user_hash) == passenger_type
```

```
class Passenger:
```

```
    def __init__(self, passenger_type: PassengerType, user_id: str):
```

```
        self.passenger_type = passenger_type
```

```
        self.user_hash = hash_id(user_id)
```

```
        # 🔍 Verify eligibility
```

```
        self.is_verified = verify_eligibility(self.user_hash, passenger_type)
```

```
    def get_discount(self):
```

```
        # ! Only apply discount if verified
```

```
        if self.is_verified:
```

```
            return DISCOUNTS[self.passenger_type]
```

```
        return 0.0
```

```
class Ticket:
```

```
    def __init__(self, ticket_type: TicketType, quantity):
```

```
        if not isinstance(quantity, int):
```

```
            raise TypeError("Quantity must be an integer.")
```

```
        if quantity <= 0:
```

```
            raise ValueError("Quantity must be greater than 0.")
```

```
        self.ticket_type = ticket_type
```

```
        self.quantity = quantity
```

```
    def get_base_price(self):
```

```
        return BASE_PRICES[self.ticket_type] * self.quantity
```

```
class Order:
```

```
    def __init__(self, passenger: Passenger):
```

```
        self.passenger = passenger
```

```
        self.tickets = []
```

```
    def add_ticket(self, ticket: Ticket):
```

```
        self.tickets.append(ticket)
```

```
    def calculate_total(self):
```

```
        total = sum(ticket.get_base_price() for ticket in self.tickets)
```

```
        discount = self.passenger.get_discount()
```

```
        final_total = total * (1 - discount)
```

```
        return round(final_total, 2)
```

```
# ---- Example usage ----

def main():
    try:
        # Simulate adding a verified student
        test_user_id = "student@email.com"
        test_hash = hash_id(test_user_id)
        ELIGIBILITY_DB[test_hash] = PassengerType.STUDENT

        passenger = Passenger(PassengerType.STUDENT, test_user_id)

        order = Order(passenger)
        order.add_ticket(Ticket(TicketType.SINGLE, 2))
        order.add_ticket(Ticket(TicketType.BUNDLE, 1))

        total_price = order.calculate_total()

        print(f"Total price: ${total_price}")
        print(f"Verified: {passenger.is_verified}")

    except (ValueError, TypeError) as e:
        print(f"Error: {e}")

if __name__ == "__main__":
    main()
```

What this fixes

✓ No fake discounts

- A user **cannot just claim** to be a student/senior
- Must pass verification (`is_verified == True`)

✓ PI still protected

- Only **hashed IDs** used
- No raw emails or personal data stored

✓ Real-world ready structure

In production, `verify_eligibility()` would:

- Call a **government/student database API**
- Check **ID cards / accounts**
- Use **secure authentication**

⚠ Subtle but important point

Right now, if verification fails:

- The user still buys tickets
- They just **don't get the discount**

That's usually the correct real-world behavior.

If you want, I can upgrade this further with:

- ↳ Expiring eligibility (e.g., student status expires)
- ↳ Multi-user concurrent simulation (to match "busy times" requirement)
- ↳ Unit tests (often required in assignments)

Just tell me 👍

